

KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY



Project Title: WorkVault — Freelance Marketplace with Milestone Escrow

Course Title: Web Programming Laboratory

Project Type: Full-Stack Web Application

SUBMITTED TO

Ehsanul Karim Talha
Lecturer
Department of Computer Science and Engineering
Khulna University of Engineering & Technology (KUET)

Mr. Kazi Saeed Alam
Assistant Professor
Department of Computer Science and Engineering
Khulna University of Engineering & Technology (KUET)

SUBMITTED BY

Name: Ahnaf Tajwar Sadi
Roll: 2207104
Section: B
Department: Computer Science and Engineering
Year: 3rd
Semester: 1st

Submission Date: 20/07/2026

GitHub Repository: https://github.com/ATSadi/Freelance_Marketplace

Abstract

WorkVault is a full-stack freelance marketplace web application designed to digitize the complete life cycle of freelance work — from posting a project and receiving competitive proposals, through milestone-based delivery and escrow-style payment accounting, to reviews, invoices and administrative mediation. The system was developed using Laravel 13 (PHP 8.3), Blade templates, Tailwind CSS and Alpine.js, with PostgreSQL as the relational database. It provides secure, role-based access for three kinds of users — clients, freelancers and administrators — and covers registration, authentication, project publication, proposal bidding, hiring, milestone submission and approval, a mock escrow transaction ledger, Stripe Checkout funding, order tracking, project messaging, dispute management, two-sided reviews, invoice generation, freelancer wallets with withdrawal processing, and a complete administration panel.

This Web Laboratory report presents the project from a web-engineering perspective. It covers the system requirements, the Model–View–Controller (MVC) architecture, routing and middleware implementation, the core web modules with interface evidence, external payment API integration, security measures, testing methodology and results, and version control practices. In addition, the report documents the database design through an Entity–Relationship (ER) diagram and a relational schema diagram, giving a comprehensive overview of the system's architecture and implementation.

1. Introduction

Freelance engagements are traditionally coordinated through a patchwork of disconnected tools: job posts on one site, negotiation over email, files over chat applications, and payments through personal transfers. This fragmentation creates two chronic problems. First, clients have no structured way to verify that work is progressing before money changes hands, so payment either happens too early (risking loss) or too late (starving the freelancer). Second, freelancers have no verifiable evidence that a client has actually allocated funds for the work, and no neutral mechanism to resolve conflicts when the two sides disagree about a deliverable.

WorkVault addresses these problems by placing the entire engagement inside a single browser-based platform. Every contract is decomposed into milestones with explicit amounts and due dates. When a client hires a freelancer, funds for each milestone are recorded as held in an internal escrow ledger; they are released only when the client approves the submitted work, or refunded when an order is cancelled or a dispute is resolved in the client's favour. Communication happens in a per-project message thread, disagreements are escalated to an administrator through a formal dispute workflow, and every monetary movement is recorded as an auditable transaction row. The result is a transparent, reviewable trail for both parties from the first proposal to the final review.

The application is deployed as a Laravel web application. Users access role-specific portals through standard HTTP routes. The presentation layer uses Blade and Tailwind CSS for responsive layouts; the application layer uses controllers, form requests, policies and middleware; the domain layer encapsulates escrow accounting in a dedicated service class; and the data layer uses PostgreSQL accessed through the Eloquent ORM.

1.1 Scope

- Client and freelancer registration, authentication, dashboards and profiles.
- Project posting, searching, filtering, saving and proposal bidding.
- Hiring, automatic milestone plan generation and full order tracking (active, completed, cancelled).
- Milestone submission, approval, change requests and mock escrow settlement.
- Stripe Checkout integration for funding milestones in test mode.
- Project messaging, in-app notifications, disputes, two-sided reviews and invoices.
- Freelancer wallet with encrypted payout methods and admin-moderated withdrawals.
- Administrator management of users, projects, payments, withdrawals and disputes.

2. Project Objectives

1. Design and implement a three-tier web architecture using the Laravel MVC pattern.
2. Provide separate, secure web portals for clients, freelancers and administrators with role-based middleware.
3. Build responsive Blade interfaces with Tailwind CSS for forms, tables, dashboards and status badges.
4. Implement server-side validation, session authentication, CSRF protection and record-level authorization policies.
5. Support the complete freelance order life cycle — post, bid, hire, deliver, approve, pay and review — through web workflows.
6. Maintain a traceable escrow ledger in which every hold, release and refund is a database transaction row.
7. Integrate an external REST payment API (Stripe Checkout) including webhook signature verification.
8. Demonstrate maintainable routing, controllers, views, reusable Blade components and automated feature tests.
9. Document the supporting data model using ER and relational schema diagrams.

3. Requirements Analysis

3.1 Functional Requirements

- Users can register as a client or freelancer, log in, log out, reset passwords and manage profiles.
- Clients can create, edit and delete open projects with a title, description, budget range, deadline and category.
- Freelancers can browse open projects with keyword, category and budget filters, save projects for later and submit one proposal per project with a cover letter, proposed amount and duration.
- Clients can accept or reject proposals; acceptance assigns the freelancer, rejects competing bids atomically and generates a starter milestone plan with escrow holds.
- Freelancers can start and submit milestones with delivery notes; clients can approve or request changes; approving the final milestone completes the project automatically.
- Clients can fund milestones through Stripe Checkout; successful payment is confirmed by webhook and synchronized with the escrow ledger.
- Both participants can view active, past and cancelled orders with progress, funded/released totals and deadlines, and can cancel an in-progress order (refunding unreleased holds).
- Participants can exchange messages inside each hired project and receive in-app notifications for all significant events.
- Either participant can open a dispute; administrators review and resolve it with an optional financial action (release, refund or cancel with refunds).
- After completion, both sides can publish exactly one star rating and comment about each other; paid milestones produce printable invoices.
- Freelancers can register encrypted payout methods, view wallet balances and request withdrawals; administrators approve, reject or mark them paid.
- Administrators can search and suspend users, moderate project status, and audit the complete escrow and Stripe payment ledgers.

3.2 Non-Functional Requirements

- **Security:** bcrypt password hashing, CSRF tokens, session regeneration, role middleware, per-record policies, encrypted bank account fields, and Stripe webhook signature verification.
- **Usability:** clear role-aware navigation, status badges, flash feedback messages and responsive layouts for desktop and tablet widths.
- **Reliability:** database transactions around hiring, escrow movement, dispute resolution and withdrawal creation; idempotent escrow operations that cannot double-release or double-refund.
- **Maintainability:** MVC separation, named routes, form request classes, policy classes, a dedicated escrow service and reusable Blade components.

- **Compatibility:** modern desktop browsers (Chrome, Edge, Brave).

4. Web Architecture

The application follows a classic three-layer web architecture built on Laravel's MVC conventions.

4.1 Presentation Layer

Laravel Blade templates render HTML for all three portals. Tailwind CSS provides responsive layouts, form controls, cards, tables and status badges, compiled through Vite. Alpine.js powers small interactive behaviours such as dropdown menus and mobile navigation. Shared layouts (`layouts/app.blade.php` for authenticated users and `layouts/guest.blade.php` for visitors) define the navigation chrome, and reusable components include the application logo, primary/secondary buttons, text inputs, flash-status banners and status badges for projects, proposals, milestones, disputes and transactions.

4.2 Application Layer

HTTP requests enter through `routes/web.php` and are dispatched to controllers. Custom middleware protects role-specific route groups: the `role` middleware restricts client, freelancer and admin prefixes, and a global `EnsureUserIsActive` middleware logs out suspended accounts on their next request. Controllers validate input through dedicated Form Request classes, authorize actions through policies, interact with Eloquent models and the escrow service, dispatch notifications and return Blade views or redirects with flash messages.

4.3 Data Layer

PostgreSQL stores users, profiles, projects, proposals, milestones, transactions, disputes, messages, saved projects, reviews, Stripe payments, payout methods, withdrawal requests, notifications and session data. Laravel communicates through the `pdo_pgsql` driver and the Eloquent ORM. Foreign keys with explicit cascade, restrict and null-on-delete rules preserve referential integrity. The detailed relational design appears in Section 8.

4.4 Request Flow

```
Browser
-> Named Route          (routes/web.php)
    (+ middleware: auth, verified, role:*, active)
-> Controller           (Form Request validation / Policy authorization)
-> Eloquent Model or EscrowService::hold() / release() / refund()
-> PostgreSQL           (transactional read / write)
<- Result set / model instances
<- Blade View or Redirect + flash status
<- HTML Response
```

4.5 Route Groups

- **Public:** / (landing page), /stripe/webhook (signature-verified, CSRF-exempt).
- **Guest authentication:** /register, /login, /forgot-password, /reset-password.
- **Client (role:client):** /client/dashboard, /client/projects/*, nested milestone management, proposal accept/reject.
- **Freelancer (role:freelancer):** /freelancer/dashboard, /freelancer/projects/browse, /freelancer/proposals, milestone start/submit.
- **Admin (role:admin):** /admin/dashboard, users, projects, payments, withdrawals and dispute moderation.
- **Shared authenticated:** orders, wallet, messages, notifications, disputes, reviews, saved projects, freelancer directory, invoices and profile management, protected by ownership and participant checks.

5. Technology Stack

- Backend framework: Laravel 13 (PHP 8.3)
- Templating: Blade
- Styling: Tailwind CSS with the Vite build pipeline
- Client-side interactivity: Alpine.js
- Database: PostgreSQL (via pdo_pgsql and Eloquent ORM)
- External API: Stripe Checkout and Stripe Webhooks (stripe-php SDK)
- Authentication scaffold: Laravel Breeze
- Testing: PHPUnit 12 feature and unit suites
- Version control: Git / GitHub
- Local server: php artisan serve (development)

6. Implementation of Web Modules

6.1 Registration and Role Selection

The registration page collects name, email, role and password with confirmation. Laravel validates required fields, e-mail uniqueness and password strength, hashes the password, stores the user with the selected role and regenerates the session before redirecting to the appropriate dashboard. The role selector is a two-card control that explains what each role can do, so a new visitor immediately understands the difference between hiring and working. Administrators are seeded rather than self-registered, which keeps the privileged role out of the public sign-up path.

6.2 Login

The login form posts email and password to the authentication controller. The application verifies the password hash, regenerates the session and redirects the user to the dashboard that matches

their stored role. Login attempts are rate-limited to five per throttle key to slow brute-force attacks, and accounts that an administrator has suspended are rejected with an explanatory message. A quick-login panel fills in seeded demonstration credentials for each role so evaluators can move between the client, freelancer and admin perspectives quickly.

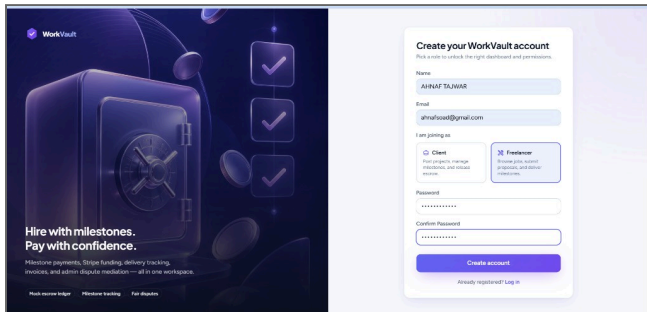


Figure 1: Registration interface with role selection

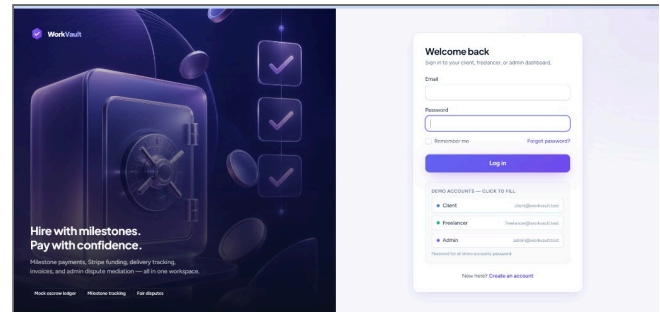


Figure 2: Login page with quick-login helpers

6.3 Public Landing Page

Unauthenticated visitors see a marketing landing page that summarizes the platform's value: milestone-based hiring, a mock escrow ledger, delivery tracking, invoices and dispute mediation. Prominent calls to action lead to registration as a client or as a freelancer.

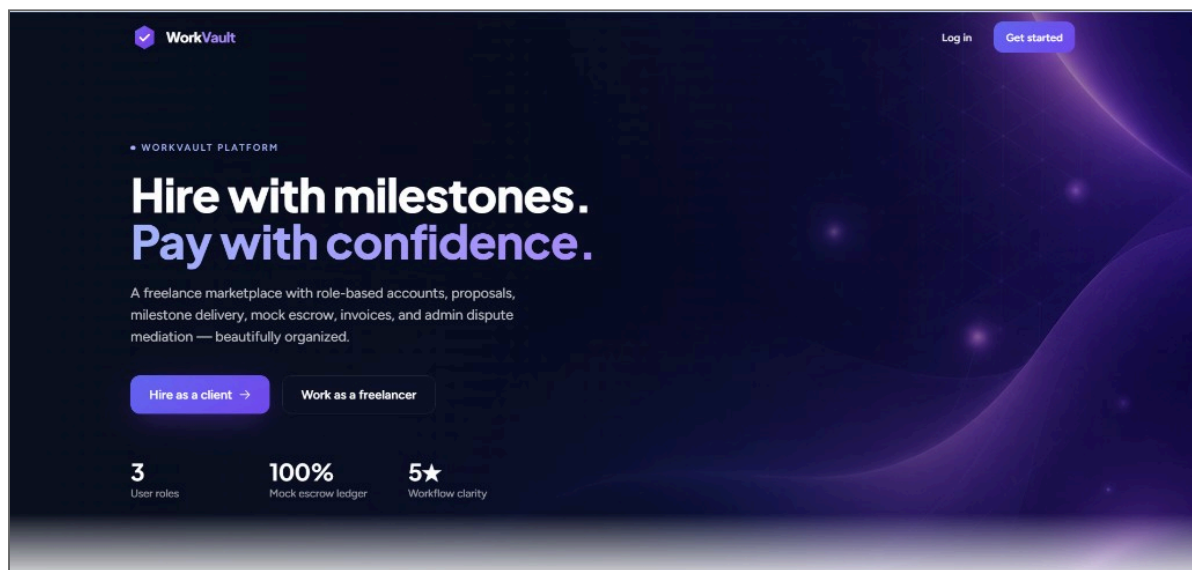


Figure 3: Public landing page

6.4 Client Dashboard and Project Posting

The client dashboard shows the completed company profile, total and active project counts, quick links to orders, projects and transactions, and a list of in-progress contracts with their assigned freelancers. From here the client posts new work by supplying a title, a detailed description, minimum and maximum budget, a deadline and a category. All fields are validated server-side; the budget maximum must not be lower than the minimum and the deadline must be a future date. Open projects can be edited or withdrawn, but once a freelancer is hired the contract can no longer be deleted — it must be completed or formally cancelled so that the financial history is preserved.

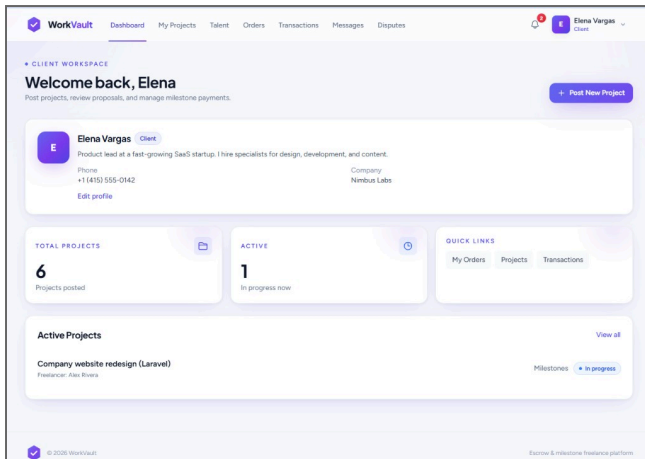


Figure 4: Client dashboard with statistics and active contracts

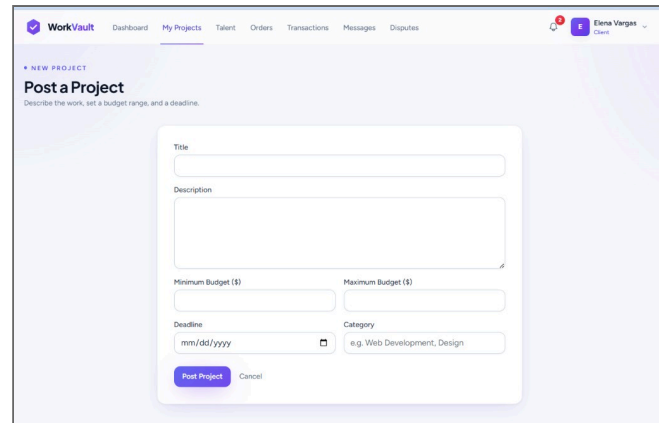


Figure 5: Project posting form with budget range and deadline

6.5 Freelancer Dashboard and Profile

Freelancers maintain a work profile containing a biography, skills, hourly rate, phone number and profile photo. The dashboard summarizes submitted proposals, active projects and quick links to orders, wallet and proposals. Profile completeness is encouraged with an inline banner for new accounts, because the same profile information is displayed publicly in the talent directory that clients browse when choosing whom to invite or hire.

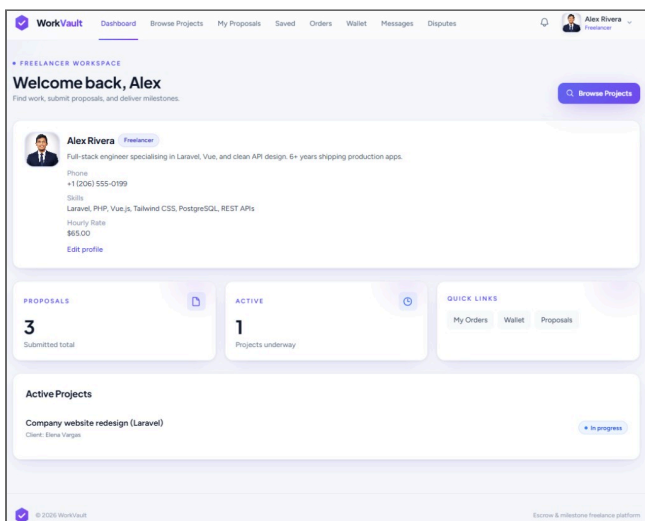


Figure 6: Freelancer dashboard with profile and active work

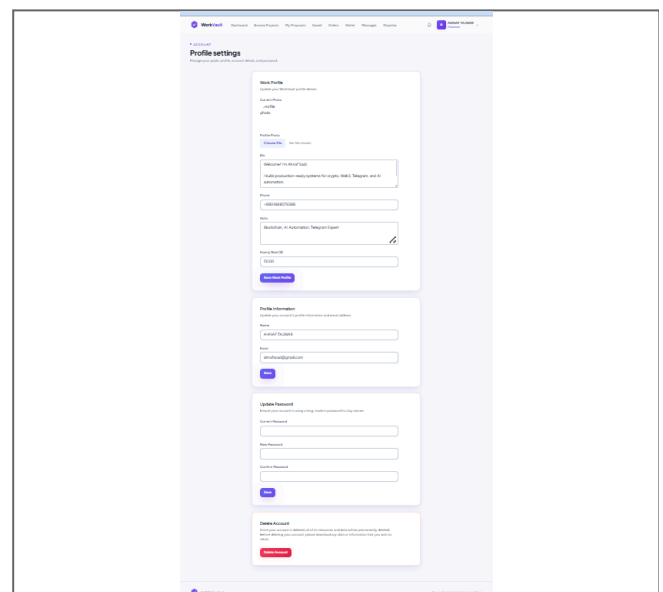


Figure 7: Profile settings — work profile, account, password

6.6 Browsing, Searching and Saving Projects

The marketplace page lists open projects as cards showing category, description excerpt, budget range and deadline. A filter bar supports keyword search, category selection, minimum/maximum budget bounds and sort order; the query is executed server-side with parameterized conditions. Each card carries a bookmark control backed by a many-to-many pivot table, letting a freelancer build a shortlist of interesting briefs and return to them later from the Saved Projects page.

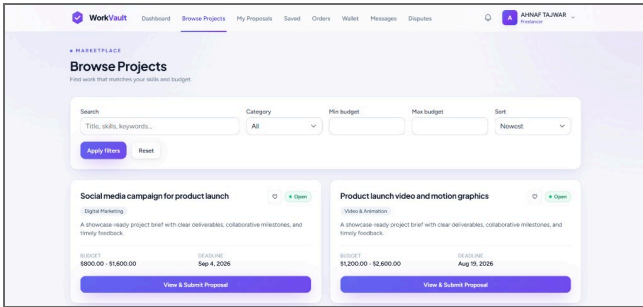


Figure 8: Project marketplace with search and filters

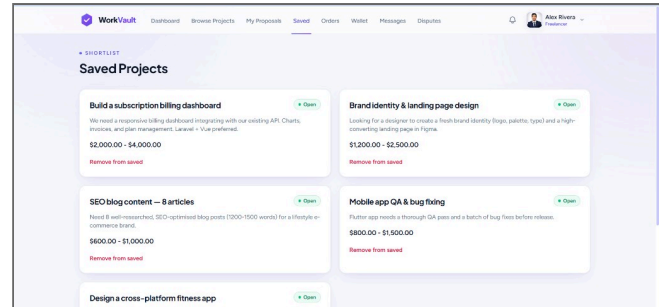


Figure 9: Saved-project shortlist

6.7 Freelancer Directory

Clients can discover talent through a public directory that lists freelancer profiles with skills, hourly rates and average review ratings. A search box matches names and skill keywords. Ratings are aggregated from the two-sided review system described in Section 6.13, so the directory reflects genuine completed-project feedback rather than self-reported claims.

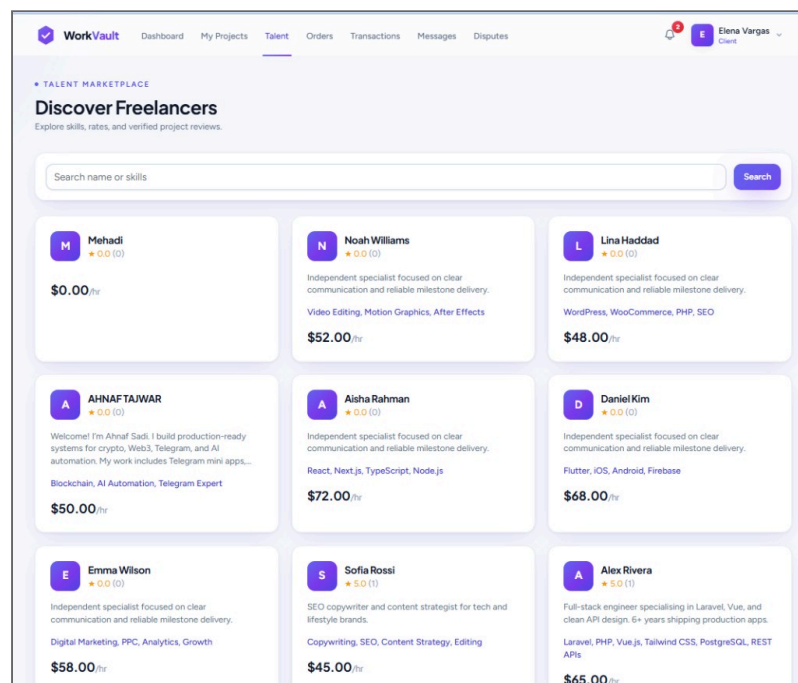


Figure 10: Freelancer directory with skills, rates and review ratings

6.8 Proposals and Hiring

Opening a project brief shows the full description, budget, deadline and client identity, followed by a proposal form with a cover letter, proposed amount and estimated duration in days. A database uniqueness constraint on the (project, freelancer) pair guarantees one bid per freelancer per project. When the client accepts a bid, a single database transaction rejects all competing proposals, assigns the freelancer, moves the project to *in progress*, generates a three-stage starter milestone plan (discovery and kickoff, main delivery, revisions and handoff — split 40% / 35% / 25% of the agreed amount) and records an escrow hold for each milestone. The client is redirected straight to the new order page so that delivery can begin without any empty intermediate state.

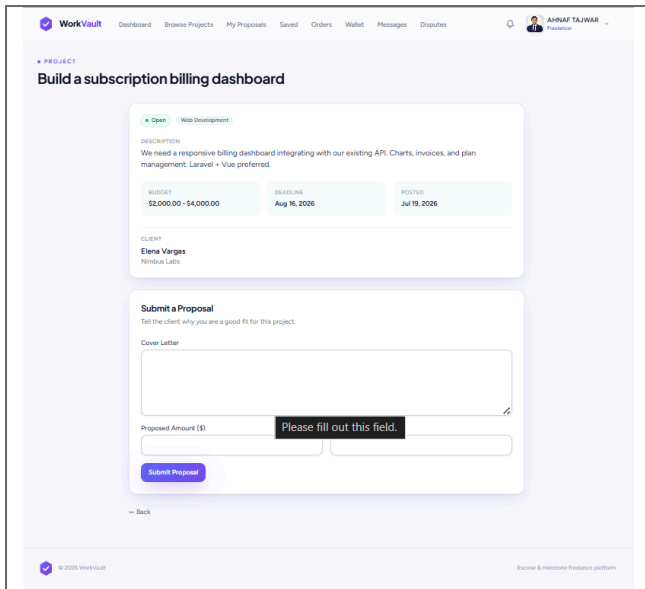


Figure 11: Project brief with proposal form and validation

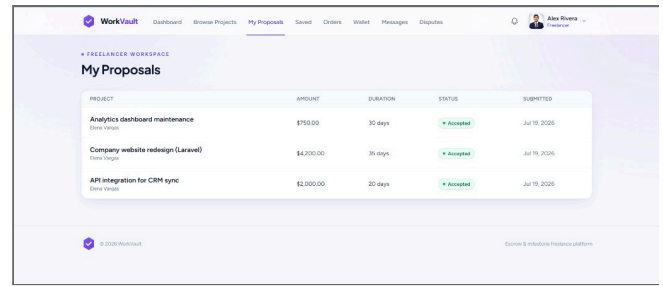


Figure 12: Freelancer proposal history and states

6.9 Orders Workspace

Every hired project becomes an order. The orders workspace groups them into three tabs — active, past and cancelled — with per-tab counts. Each card shows the counterpart's name, milestone completion progress, the next due milestone and the contract value. This gives both sides one place to answer the everyday questions: what is running now, what has been delivered before, and what was called off. An in-progress order can be cancelled by either participant after a confirmation prompt; cancellation refunds every escrow hold that has not yet been released and moves the order to the cancelled tab.

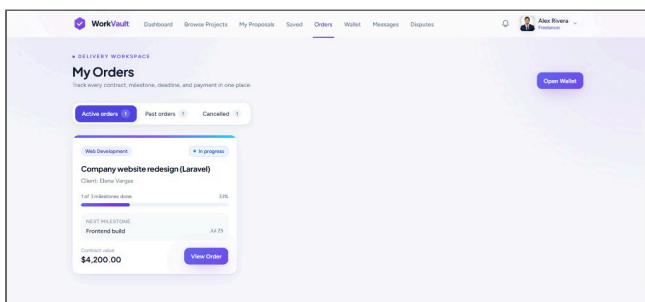


Figure 13: Orders workspace — active orders tab

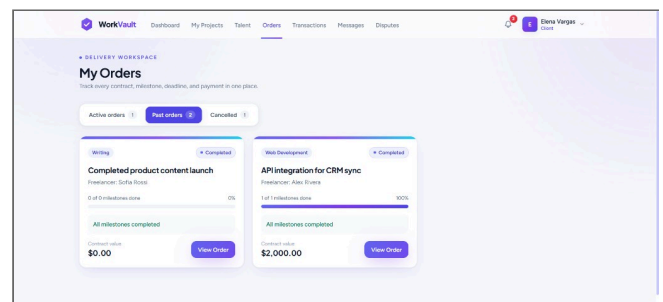


Figure 14: Past-orders archive with progress and values

6.10 Milestones, Escrow and the Order Detail Page

The order detail page is the operational centre of a contract. Its header summarizes order status, contract value, funded versus released amounts and the final deadline, alongside actions to message the counterpart, open a dispute or cancel the order. The milestone list shows each stage with its amount, due date, delivery notes and status badge. A milestone moves through five states: *pending* → *in progress* → *submitted* → *approved* → *paid*. The freelancer starts a milestone and submits it with delivery notes; the client either approves it — which triggers the escrow release and

marks it paid — or requests changes, returning it to the freelancer with feedback. When the last milestone is paid the project is automatically marked completed.

Escrow accounting is implemented by a dedicated `EscrowService` whose operations are idempotent: a milestone can hold funds only once, and once released it can never be refunded (or vice versa). Editing a pending milestone's amount synchronizes the corresponding hold, and cancelling an order or resolving a dispute refunds precisely those holds that remain open. Every movement is stored as a transaction row with payer, payee, amount, type and description, forming a complete audit trail.

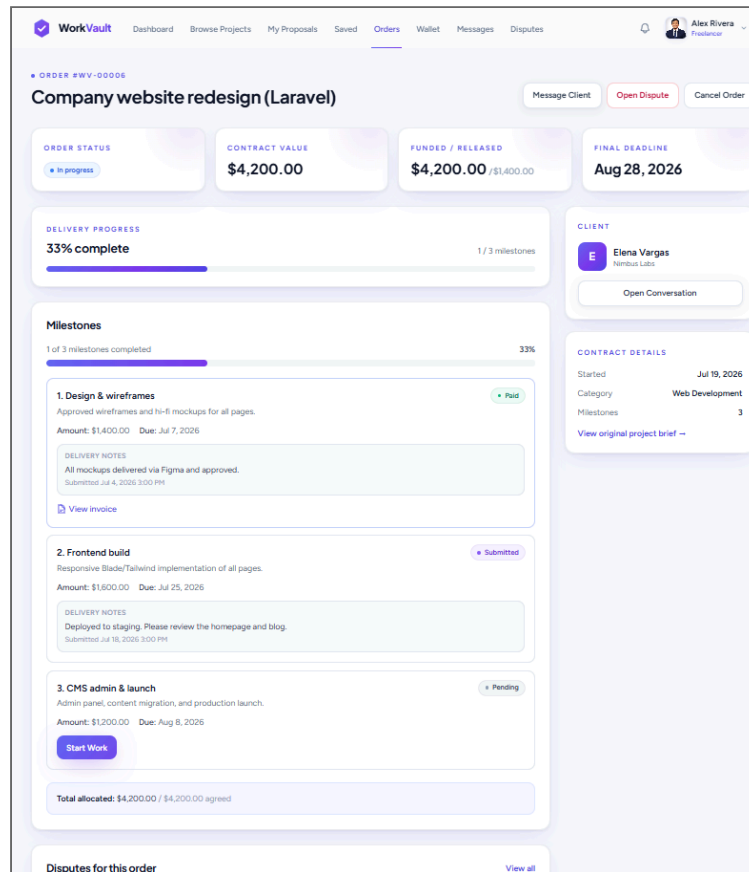


Figure 15: Order detail page — status header, milestone pipeline, funded/released totals and actions

6.11 Invoices

Every paid milestone generates a printable invoice showing the billed client, the paid freelancer, the project, the milestone description and the settled amount. The invoice can be printed or saved as PDF through the browser's print dialog, or downloaded as a standalone HTML file. Access is limited to the two participants of the project.

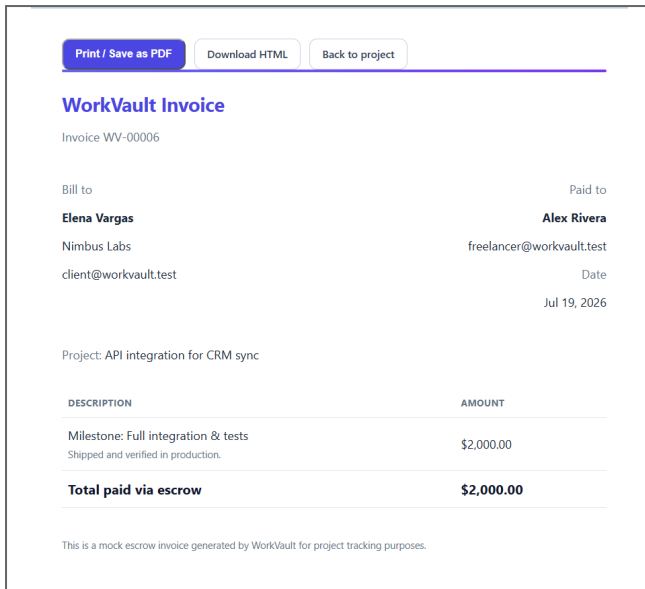


Figure 16: Milestone invoice with print controls

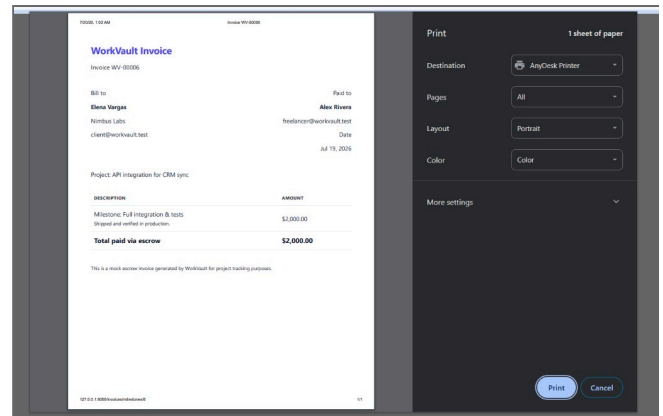


Figure 17: Browser print preview of an invoice

6.12 Project Messaging and Notifications

Each hired project carries a private two-person conversation. The inbox lists conversations with the counterpart's name, the latest message preview and an unread count; the thread view renders messages chronologically with sender attribution and timestamps, marking incoming messages read when the thread is opened. Participant checks in the controller prevent any third account from reading or posting into a conversation. Parallel to messaging, Laravel database notifications alert users about proposal submissions and acceptances, milestone submissions and approvals, new messages, opened disputes and withdrawal status changes; a bell menu in the navigation shows the unread count and links each notification to its target page.

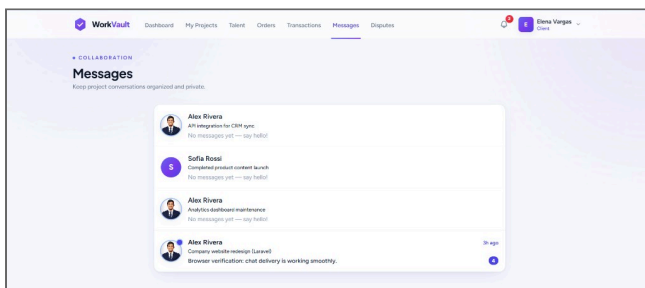


Figure 18: Conversation inbox with unread badges

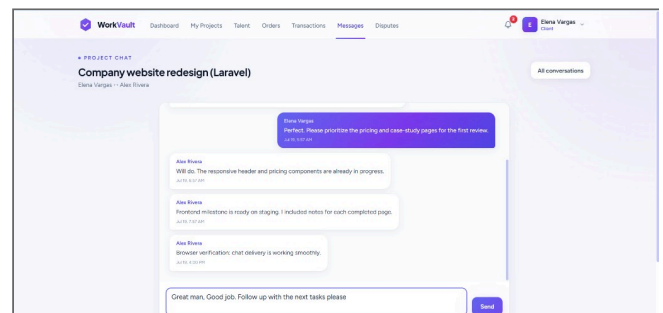


Figure 19: Project chat thread with composer

6.13 Disputes and Two-Sided Reviews

When collaboration breaks down, either participant can open a dispute against the other for an in-progress project, optionally attaching the specific milestone in question and describing the problem. The dispute record moves through *open* → *under review* → *resolved* / *dismissed*. Administrators investigate the evidence, add decision notes and choose a financial action as part of the resolution: release the disputed milestone's escrow to the freelancer, refund it to the client, refund all open holds and cancel the project, or close the dispute without moving money. The

status change and the financial action are executed inside one database transaction so the ledger can never disagree with the verdict.

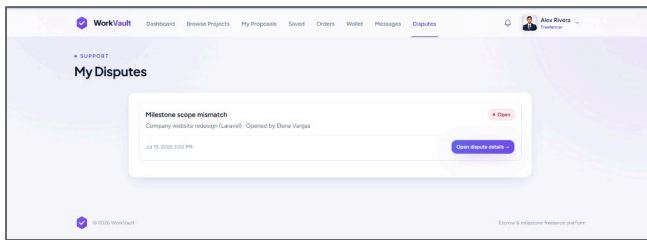


Figure 20: Participant dispute list

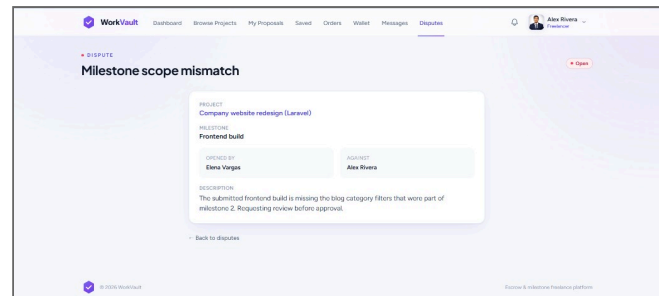


Figure 21: Dispute detail with milestone context and parties

After a project completes, each participant can publish exactly one review of the other — a one-to-five-star rating and a written comment — enforced by a unique database constraint. Reviews are displayed on the completed order page and aggregated into the average ratings shown in the freelancer directory, creating an incentive for professional behaviour on both sides of the marketplace.

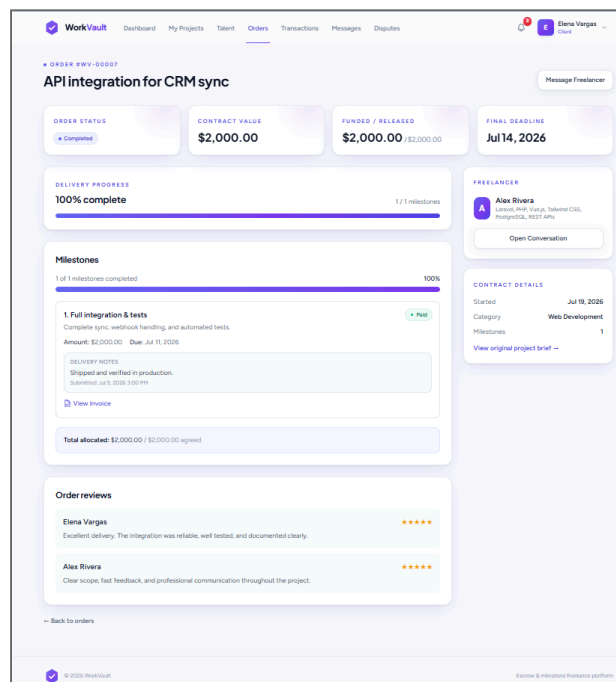


Figure 22: Completed order with two-sided reviews and invoice access

6.14 Freelancer Wallet and Withdrawals

The wallet page computes four figures from the transaction ledger: total earned (completed escrow releases), pending release (funded milestones not yet approved), the amount reserved by pending or approved withdrawals, and the resulting available balance. Freelancers register a payout destination — bank account or mobile wallet — whose account and routing numbers are stored with Laravel's encrypted model casts, exposing only the last four digits in any interface. A withdrawal request validates the amount against a freshly computed balance inside a row-level

database lock, preventing two concurrent requests from overdrawing the wallet. The request then flows through the states *pending* → *approved* → *paid* (or *rejected* at either stage, which frees the reserved amount), with each transition performed by an administrator and reported back to the freelancer by notification.

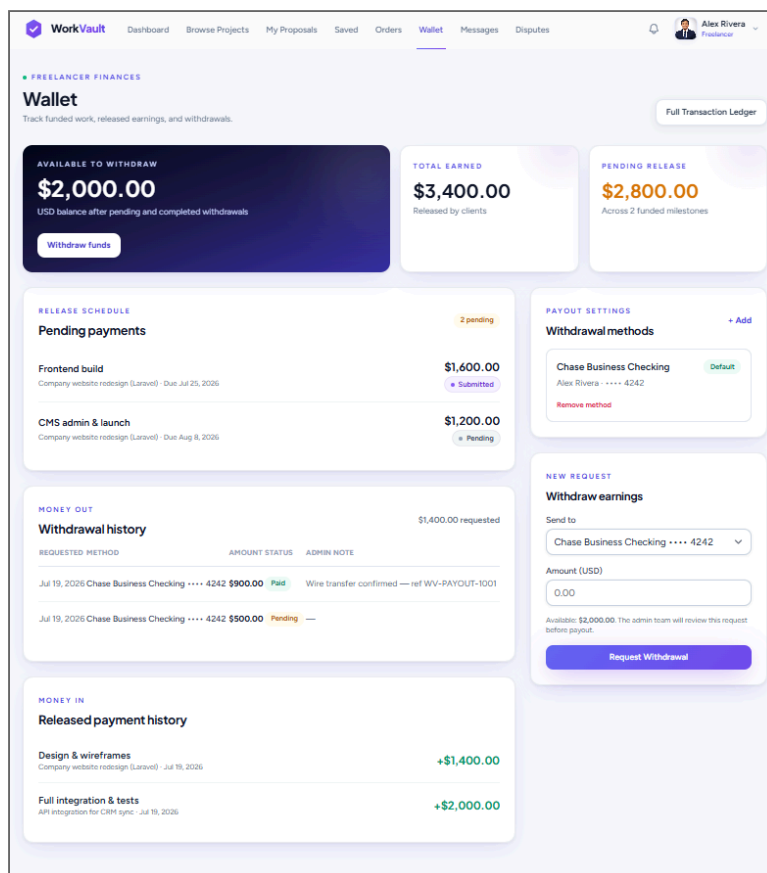


Figure 23: Wallet — balances, release schedule, payout method, withdrawal request and history

6.15 Administration Panel

Administrators operate the platform from a dedicated panel. The dashboard summarizes user counts by role, the project pipeline, open disputes, total escrow released and pending withdrawals, with recent-activity panels for disputes and projects. The users module supports search and role filtering and lets the administrator suspend or reactivate any account; a suspended user is refused at login and, if already signed in, is logged out by middleware on the next request. The projects module allows inspection and status moderation — including forced cancellation, which refunds open escrow holds. The payments module presents the full escrow ledger alongside Stripe funding attempts, and the withdrawals module processes freelancer payout requests with an enforced state machine and free-text administrative notes.

7. External API Integration (Stripe)

WorkVault integrates the Stripe REST API so that a client can fund a milestone with a real card flow in test mode. Pressing the fund button creates a Stripe Checkout Session for the milestone amount and redirects the browser to Stripe's hosted payment page. On success, Stripe redirects back to the application and simultaneously delivers a `checkout.session.completed` event to the webhook endpoint. The webhook handler verifies the event signature with the shared webhook secret before trusting any payload, marks the corresponding `stripe_payments` record as paid, and synchronizes the internal escrow ledger by recording the hold for that milestone. Because the ledger hold operation is idempotent, receiving both the redirect confirmation and the webhook cannot double-fund a milestone.

```
$session = \Stripe\Checkout\Session::create([
    'mode'          => 'payment',
    'line_items'    => [[
        'price_data' => [
            'currency'      => 'usd',
            'unit_amount'   => (int) round($milestone->amount * 100),
            'product_data' => ['name' => 'Milestone: '.$milestone->title],
        ],
        'quantity'      => 1,
    ]],
    'success_url' => route('stripe.success', $payment).'?session_id={CHECKOUT_SESSION_ID}',
    'cancel_url'  => route('orders.show', $milestone->project),
]);
```

Missing or invalid Stripe credentials are handled gracefully: the controller reports a configuration error instead of failing, so the rest of the platform remains usable when payment keys are not present in the environment.

8. Database Design

Although this report emphasizes the web application, a clear data model underpins every page. PostgreSQL stores fifteen domain tables besides Laravel's infrastructure tables (sessions, cache, jobs and notifications). The conceptual and physical designs are shown below.

8.1 Entity–Relationship Diagram

The ER diagram shows users at the centre of the identity domain, projects as the hub of the contract domain, and milestones as the unit of delivery and payment. A user in the client role owns many projects; a user in the freelancer role is assigned to many projects and submits many proposals. Each project has many milestones, transactions, messages, disputes and reviews, and may be saved by many freelancers through the `saved_projects` bridge. Milestones connect to Stripe payment attempts and to the escrow transactions that hold, release or refund their

amounts. Payout methods and withdrawal requests belong to freelancers and are processed by administrators.

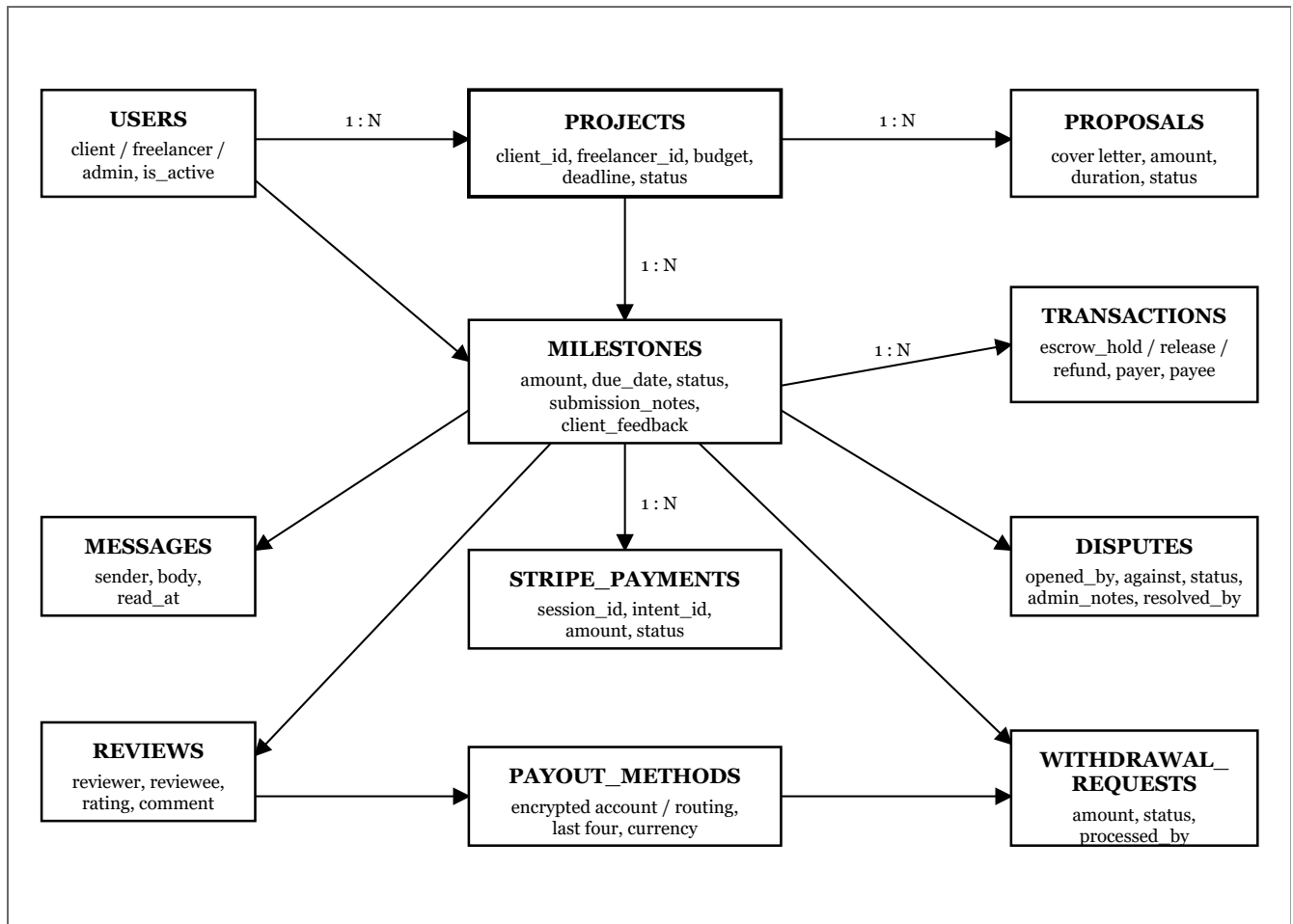


Figure 30: Entity-relationship diagram of the WorkVault marketplace

8.2 Relational Schema Diagram

The schema diagram presents the physical tables with their attributes, primary keys and foreign keys as used by the Eloquent models and web controllers. Referential actions are declared explicitly in the migrations: records owned by a project (milestones, messages, proposals, reviews) cascade on delete; optional references such as a project's assigned freelancer or a dispute's resolver are set to null; and payout methods referenced by withdrawal requests are restricted from deletion to preserve the payout history. Two composite constraints deserve mention — `saved_projects` uses (user_id, project_id) as its composite primary key, and `reviews` enforces uniqueness of (project_id, reviewer_id) so each participant may review a project once.

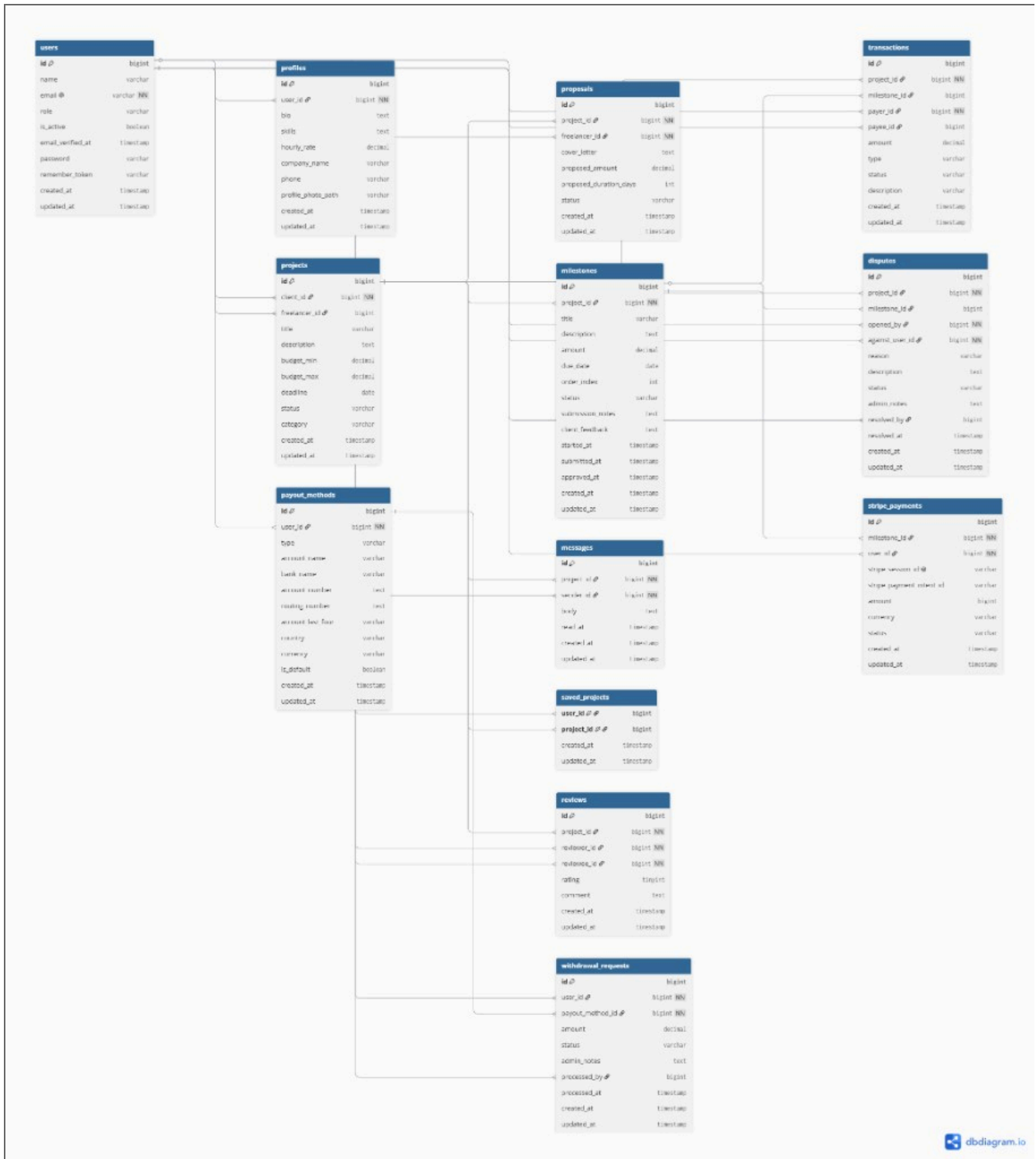


Figure 31: Relational schema diagram with primary keys, foreign keys and relationships

9. Web Security Features

- Laravel CSRF tokens on all POST / PATCH / DELETE forms; only the signature-verified Stripe webhook is exempt.
- Bcrypt password hashing for all accounts, with current-password confirmation before sensitive profile changes.

- Session regeneration after registration and login; session invalidation and token regeneration at logout.
- Login rate limiting (five attempts per throttle key) against brute-force attacks.
- Separate `role:client`, `role:freelancer` and `role:admin` middleware guarding each portal's routes.
- A global active-user middleware that terminates the session of any account suspended by an administrator.
- Policy classes for projects, proposals, milestones and disputes; participant checks for orders, messages, invoices and reviews, so every query is scoped to the authenticated user's records.
- Form Request validation for every significant mutation, including image type/size validation on profile photo uploads.
- Encrypted storage of payout account and routing numbers via Eloquent encrypted casts; interfaces show only the last four digits.
- Database transactions and row-level locks around hiring, escrow settlement, dispute resolution and withdrawal balance checks.
- Stripe webhook signature verification; server-side price calculation from the stored milestone amount prevents forged totals.
- Secrets (database credentials, Stripe keys) kept in `.env` and never committed to version control.

10. Testing and Results

The project carries an automated PHPUnit suite of 37 tests (103 assertions) spanning authentication, authorization boundaries and the financial workflows. All 37 tests pass against the configured test database. Representative scenarios are listed below.

Area	Representative verification	Result
Authentication	Registration with role selection, login, logout, password reset and confirmation screens	Passed
Suspension	Suspended users are refused at login and logged out on their next authenticated request	Passed
Authorization	Non-participants cannot open another user's order details or project messages	Passed
Marketplace	Saving/unsaving projects, sending project messages, publishing one review per participant	Passed
Hiring	Accepting a proposal rejects competitors, creates starter milestones with escrow holds and opens the order	Passed
Escrow	Order cancellation and dispute resolution refund exactly the unreleased holds	Passed
Wallet	Withdrawals above the available balance are rejected; admin approval transitions are enforced	Passed
Project integrity	Hired projects cannot be deleted by clients	Passed
Stripe	Checkout fails gracefully without configured keys; webhooks require a valid signature	Passed

In addition, the following manual tests were performed in a local browser environment against the seeded PostgreSQL database:

1. Register a new freelancer and confirm redirect to the freelancer dashboard.
2. Log in with each demo role; reject invalid passwords.
3. Post a project as a client and verify it appears in the freelancer marketplace with correct filters.
4. Submit a proposal, accept it as the client, and verify the starter milestones and escrow holds on the order page.
5. Start, submit and approve a milestone; verify escrow release, invoice generation and progress update.
6. Exchange messages between the two participants and verify unread counts and notifications.
7. Open a dispute, resolve it as admin with a refund action, and verify the ledger entries.
8. Request a withdrawal as freelancer, approve and mark it paid as admin, and verify wallet balances.
9. Suspend a logged-in user as admin and verify forced logout on the user's next request.
10. Verify client A cannot see client B's orders, messages or transactions.

11. Discussion and Future Work

11.1 Strengths

- Complete two-sided marketplace journey from project discovery to reputation, in one consistent interface.
- Clear separation of the client, freelancer and admin experiences with strict authorization boundaries.
- Traceable financial model: every hold, release and refund is an auditable transaction row, and escrow operations are idempotent.
- Useful operational features: order tracking, wallets, invoices, notifications, dispute mediation and admin auditing.
- Strong automated coverage of the highest-risk paths (money movement and access control).

11.2 Limitations and Future Improvements

- The escrow ledger is a demonstrative accounting model; production use would require a regulated marketplace payment provider with connected accounts and real fund custody.
- Email verification routes exist from the Breeze scaffold, but enforcement via the `MustVerifyEmail` contract is not yet enabled on the user model.
- Messaging uses request/response refresh; WebSockets (Laravel Reverb) would enable real-time delivery and typing indicators.
- Milestone submissions are notes-based; file uploads with object storage and virus scanning would strengthen delivery evidence.
- Invoice download currently exports printable HTML rather than a server-generated binary PDF.
- Future work: identity verification, platform service fees, multi-currency support, richer analytics, automated browser tests and continuous integration.

12. GitHub Repository and Version Control

Source code is hosted at https://github.com/ATSadi/Freelance_Marketplace on the master branch. Development progressed through iterative commits covering the initial Laravel and PostgreSQL setup, authentication and roles, profiles, project posting and proposals, the milestone and escrow systems, disputes and the admin panel, notifications and invoices, the orders and wallet workspaces, and final UI polish. The repository layout follows standard Laravel conventions: `app/` (controllers, models, middleware, policies, requests, notifications, services), `resources/` (Blade views and assets), `database/` (migrations and seeders), `routes/`, `tests/` and `public/`.

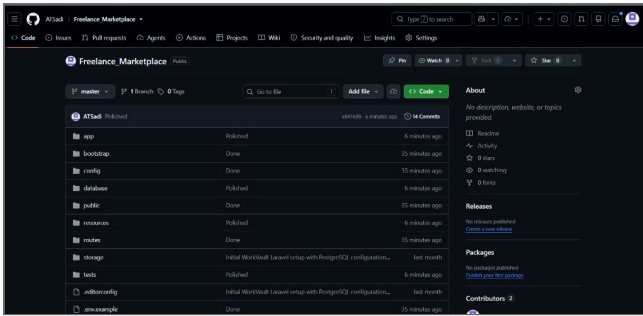


Figure 32: Public GitHub repository page

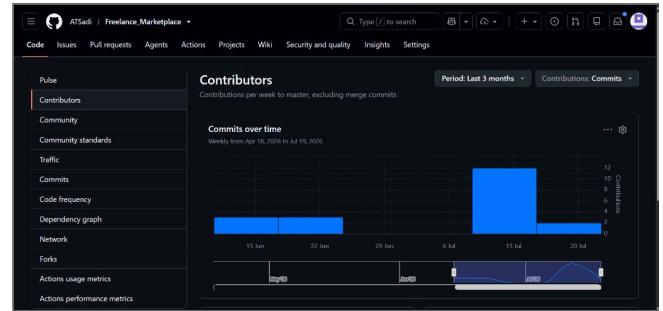


Figure 33: GitHub contribution and commit activity

13. Conclusion

WorkVault demonstrates a complete web laboratory project: MVC architecture, role-based portals, responsive Blade interfaces, secure sessions, CRUD workflows, external payment API integration, and clear navigation for real marketplace tasks. Its distinguishing feature is the financial backbone — milestones, an auditable mock escrow ledger, Stripe funding, wallets and moderated withdrawals — which turns an ordinary job board into a structured, accountable delivery platform. The ER and relational schema diagrams document the data foundation that every page depends on, and the automated test suite guards the money-movement and access-control paths that matter most. Together, Laravel and PostgreSQL deliver a practical, browser-accessible freelance marketplace suitable for further extension into production services.

14. References

1. Laravel Documentation. <https://laravel.com/docs>
2. Tailwind CSS Documentation. <https://tailwindcss.com/docs>
3. PostgreSQL Documentation. <https://www.postgresql.org/docs/>
4. Stripe Checkout Documentation. <https://docs.stripe.com/checkout>
5. Alpine.js Documentation. <https://alpinejs.dev/>
6. PHPUnit Documentation. <https://phpunit.de/documentation.html>
7. Project repository: https://github.com/ATSadi/Freelance_Marketplace